МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Національний аерокосмічний університет

«Харківський авіаційний інститут»

Факультет Інтелектуальних систем управління

Кафедра Систем управління літальними апаратами

Лабораторна робота № 8

з дисципліни «Алгоритмізація та програмування»

на тему: «Реалізація алгоритмів сортування та робота з файлами на мові С ++»

ХАІ.301. G3.319а.21 ПР

Виконав: здобувач 1 курсу, групи 319а
напряму підготовки (спеціальності)

“Електрична інженерія”

Петро ШВЕЦОВ

(ім’я і прізвище)

Прийняв: асис.,

_____Євгеній ПЯВКА

2026

МЕТА РОБОТИ

Вивчити теоретичний матеріал по алгоритмам обробки масивів на мові C++, а також бібліотеки для роботи з файлами і реалізувати оголошення, введення з файлу, обробку і виведення в файл одновимірних і двовимірних масивів на мові C ++ в середовищі Visual Studio.

ПОСТАНОВКА ЗАДАЧІ

Завдання 1. За допомогою текстового редактору створити текстовий файл «array_in_n.txt» з елементами вихідного масиву (n - номер варіанта). У програмі на C++ зчитати і перетворити цей масив відповідно до свого варіанту завдання (див. лаб.роб.No6, завд.2), ім'я файлу і необхідні змінні ввести з консолі. Вивести результати у файл «array_out_n.txt».

Array106. Дан масив розміру N. Продублювати в ньому елементи з парними номерами (2, 4, ...). Умовний оператор в циклі не використовувати.

Завдання 2. За допомогою текстового редактору створити текстовий файл «matr_in_n.txt» з елементами вихідного двовимірного масиву (n - номер варіанта). У програмі зчитати і обробити матрицю відповідно до свого варіанту завдання (лаб.роб.No7, завд.1), ім'я файлу і необхідні змінні ввести з консолі. Дописати результати в той же файл.

Matrix32. Дана цілочисельна матриця розміру M × N. Знайти номер першого з її рядків, що містять однакову кількість додатних і від’ємних елементів (нульові елементи матриці не враховуються). Якщо таких рядків немає, то вивести 0.

Завдання 3. Вивчити метод сортування відповідно до свого варіанту (див. табл. 1), проаналізувати його складність і продемонструвати на прикладі з 7-ми елементів (відповідно до свого варіанту). Реалізувати у вигляді окремої функції алгоритм сортування елементів масиву. Зчитування і виведення відсортованого масиву організувати на файлах.

23 Вибір Зростання Дійсний

Завдання 4. Введення, виведення, обробку масивів реалізувати окремими функціями з параметрами. Структурувати проєкт програми для виконання завдань 1-3 наступним чином:

main.cpp //основна функція і три функції для 3х завдань

array_utils.h //заголовки функцій для роботи з
//одновимірними масивами (завдання1,3)
array_utils.cpp //визначення функцій для роботи з
//одновимірними масивами (завдання1,3)
matrix_utils.h //заголовки функцій для роботи з
//двовимірними масивами (завдання2)
matrix_utils.cpp //визначення функцій для роботи з
//двовимірними масивами (завдання2)

Завдання 5. Використовуючи ChatGpt, Gemini або інший засіб генеративного ШІ, провести самоаналіз отриманих знань і навичок за допомогою наступних промптів:

1) «Ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4 варіантами відповіді і 5 відкритих питань. Це мають бути завдання <середнього> рівня складності на розвиток критичного та інженерного мислення. Питання мають відноситись до коду, що є у файлі звіту, і до теоретичних відомостей, що є у файлі лекції»

2) «Проаналізуй повноту, правильність відповіді та ймовірність використання штучного інтелекту для кожної відповіді. Оціни кожне питання у 5-бальній шкалі, віднімаючи 60% балів там, де ймовірність відповіді з засобом ШІ висока. Обчисли загальну середню оцінку»

3) «Проаналізуй код у звіті, і додай опис і приклади коду з питань, які є в теоретичних відомостях, але не відпрацьовано у коді при вирішенні завдань»

Проаналізуйте задані питання, коментарі і оцінки, надані ШІ. Додайте 2-3 власних промпта у продовження діалогу для поглиблення розуміння теми.

ВИКОНАННЯ РОБОТИ

Завдання 1.

Вирішення задачі Array106

Вхідні дані (ім’я, опис, тип, обмеження):

filename-ім’я файлу з масивом, string, існує

N-кількість елементів масиву, int, $2 \leq N \leq 20$

arr[i]-елементи масиву, double, довільні

Вихідні дані (ім’я, опис, тип):

arr[i]-перетворений масив, double
N-новий розмір масиву, int
файл array_out_n.txt-результат, текст
Алгоритм вирішення показано нижче
Зчитування масиву з файлу:

1. Відкрити файл для читання
2. Зчитати число N
3. Послідовно зчитати N елементів масиву
4. Закрити файл

Перетворення масиву:

1. Почати з другого елемента (індекс 1)
2. Крок циклу — 2 (щоб брати лише парні позиції)
3. Для кожного такого елемента:
    - зсунути всі елементи вправо
    - вставити копію елемента
    - збільшити розмір масиву

Запис у файл:

1. Відкрити файл для запису
2. Записати новий розмір масиву
3. Записати всі елементи
4. Закрити файл

Лістинг коду вирішення задачі наведено в дод. А (стор. 9).
Екран роботи програми показаний на рис. Б.1., Б.2.
Завдання 2.
Вирішення задачі Matrix32
Вхідні дані (ім’я, опис, тип, обмеження):
filename-ім’я файлу, string, існує
M-кількість рядків, int, 2–20
N-кількість стовпців, int, 2–20
matr[i][j]-елементи матриці, int, довільні
Вихідні дані (ім’я, опис, тип):

result-номер рядка або 0, int
файл-допис результату, текст
Алгоритм вирішення показано нижче
Зчитування матриці:

1. Відкрити файл
2. Зчитати M і N
3. Зчитати M×N елементів по рядках
4. Закрити файл

Обробка:

1. Для кожного рядка:
   - порахувати додатні (pos)
   - порахувати від'ємні (neg)
2. Якщо pos == neg і не 0:
   - повернути номер рядка
3. Якщо немає — повернути 0

Запис:

1. Відкрити файл у режимі допису
2. Записати результат
3. Закрити файл

Лістинг коду вирішення задачі наведено в дод. А (стор. 9).
Екран роботи програми показаний на рис. Б.3.
Завдання 3.
Приклад виконання алгоритму сортування вибором:
Розглянемо масив із 7 елементів:
7 3 5 2 9 1 4
Крок 1
Знаходимо мінімальний елемент → 1
Міняємо з першим:
1 3 5 2 9 7 4
Крок 2
Мінімум серед решти → 2

1 2 5 3 9 7 4

Крок 3

Мінімум → 3

1 2 3 5 9 7 4

Крок 4

Мінімум → 4

1 2 3 4 9 7 5

Крок 5

Мінімум → 5

1 2 3 4 5 7 9

Результат:

1 2 3 4 5 7 9

Зчитування масиву:

Початок

↓

Відкрити файл

↓

Зчитати N

↓

Цикл (i=0..N-1)

↓

Зчитати arr[i]

↓

Закрити файл

↓

Кінець

Сортування вибором:

Початок

↓

i = 0

↓

Знайти minIndex

↓

Обмін arr[i] ↔ arr[min]

↓

i++
↓
i < N-1 ?
↓
(так → повтор)
(ні → кінець)

Екран роботи програми показаний на рис. Б.4.

Завдання 5.

Тестові питання

1.A
2.C
3.C
4.B
5. B

Відкриті питання:

6.Бо потрібно працювати лише з елементами з парними номерами (2,4,6…), що відповідає індексам 1,3,5… у 0-індексації.

7.Послідовний перегляд кожного рядка матриці з підрахунком додатних і від'ємних елементів та порівняння їх кількості.

8.Бо для кожного елемента виконується пошук мінімального серед решти елементів, що дає вкладені цикли.

9.Дозволяє працювати з великими обсягами даних, автоматизує тестування та зберігає результати виконання програми.

10.Можливе перезаписування даних, складність контролю індексів та необхідність акуратного зсуву елементів.

ВИСНОВКИ

Було вивчено методи обробки одновимірних і двовимірних масивів, а також роботу з файлами у мові C++. Закріплено на практиці реалізацію алгоритму сортування вибором та структуризацію програми на окремі модулі. Отримано навички роботи з введенням і виведенням даних з файлів, а також аналізу алгоритмів.

ДОДАТОК А
Лістинг коду програми

```
#include <iostream>
#include <string>
#include "array_utils.h"
#include "matrix_utils.h"

using namespace std;

void task1() {
    string inFile, outFile;
    cout << "Enter input file for array: ";
    cin >> inFile;
    cout << "Enter output file: ";
    cin >> outFile;

    int n;
    double arr[40];

    readArray(inFile, arr, n);
    duplicateEvenIndexes(arr, n);
    writeArray(outFile, arr, n);
}

void task2() {
    string fileName;
    cout << "Enter matrix file: ";
    cin >> fileName;

    int m, n;
    int matr[20][20];

    readMatrix(fileName, matr, m, n);
    int result = findRow(matr, m, n);
    appendResult(fileName, result);
}

void task3() {
    string inFile, outFile;
    cout << "Enter input file for sorting: ";
    cin >> inFile;
    cout << "Enter output file: ";
    cin >> outFile;

    int n;
    double arr[20];

    readArray(inFile, arr, n);
```

```
    selectionSort(arr, n);
    writeArray(outFile, arr, n);
}

int main() {
    task1();
    task2();
    task3();
    return 0;
}
#pragma once
#include <string>

void readArray(std::string filename, double arr[], int &n);
void writeArray(std::string filename, double arr[], int n);
void duplicateEvenIndexes(double arr[], int &n);
void selectionSort(double arr[], int n);
#include "array_utils.h"
#include <fstream>

using namespace std;

void readArray(string filename, double arr[], int &n) {
    ifstream fin(filename);
    fin >> n;
    for (int i = 0; i < n; i++)
        fin >> arr[i];
    fin.close();
}

void writeArray(string filename, double arr[], int n) {
    ofstream fout(filename);
    fout << n << endl;
    for (int i = 0; i < n; i++)
        fout << arr[i] << " ";
    fout.close();
}

void duplicateEvenIndexes(double arr[], int &n) {
    for (int i = 1; i < n; i += 2) {
        for (int j = n; j > i; j--)
            arr[j] = arr[j - 1];

        arr[i + 1] = arr[i];
        n++;
        i++;
    }
}

void selectionSort(double arr[], int n) {
```

```
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex])
                minIndex = j;
        }
        double temp = arr[i];
        arr[i] = arr[minIndex];
        arr[minIndex] = temp;
    }
}

#pragma once
#include <string>

void readMatrix(std::string filename, int matr[20][20], int &m, int &n);
int findRow(int matr[20][20], int m, int n);
void appendResult(std::string filename, int result);

#include "matrix_utils.h"
#include <fstream>

using namespace std;

void readMatrix(string filename, int matr[20][20], int &m, int &n) {
    ifstream fin(filename);
    fin >> m >> n;
    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++)
            fin >> matr[i][j];
    fin.close();
}

int findRow(int matr[20][20], int m, int n) {
    for (int i = 0; i < m; i++) {
        int pos = 0, neg = 0;

        for (int j = 0; j < n; j++) {
            if (matr[i][j] > 0) pos++;
            if (matr[i][j] < 0) neg++;
        }

        if (pos == neg && pos != 0)
            return i + 1;
    }
    return 0;
}

void appendResult(string filename, int result) {
    ofstream fout(filename, ios::app);
```

```
    fout << "\nResult: " << result << endl;
    fout.close();
}
```

ДОДАТОК Б

Скрін-шоти вікна виконання програми

```
5
3.4 7.5 8.0 -5.5 0.0
```

```
7
3.4 7.5 7.5 8.0 -5.5 -5.5 0.0
```

Рисунок Б.1, Б.2.– Екран виконання програми для вирішення завдання Array106

sort_out_1.txt | matr_out_1.txt

Файл Изменить Просмотр H1

```
3 5
3 9 -8 -5 0
4 7 0 3 0
3 8 8 -2 0
Result: 1
```

Рисунок Б.3 – Екран виконання програми для вирішення завдання Matrix32

sort_out_1.txt

Файл Изменить Просмотр

```
5
-5.5 0.0 3.4 7.5 8.0
```

Рисунок Б.4 – Екран виконання програми для вирішення завдання 23

ДОДАТОК В

Діалог з ШІ для самоаналізу

Ок, уявляю себе викладачем на захисті твоєї роботи. Ось типові питання середнього рівня 👇

🔷 ✅ ТЕСТОВІ ПИТАННЯ (1–5)

1. Чому в задачі 1 не використовується допоміжний масив?

A) Це заборонено умовою завдання
B) Для економії пам’яті
C) Щоб прискорити сортування
D) Бо це неможливо в C++

---

2. Яка складність сортування вибором (Selection Sort)?

A) O(n)
B) O(log n)
C) O(n^2)
D) O(n log n)

---

3. Що відбудеться, якщо у матриці немає рядка з рівною кількістю + і -?

A) Програма аварійно завершиться
B) Повернеться -1
C) Повернеться 0
D) Повернеться NULL

---

4. Чому в задачі 2 нулі не враховуються?

A) Вони не зчитуються
B) За умовою задачі
C) Через помилку компілятора
D) Вони автоматично ігноруються мовою C++

---

5. Яка операція виконується при дублюванні елементів масиву?

A) Видалення елементів
B) Зсув елементів вправо
C) Сортування
D) Перетворення типів

🔷 🧠 ВІДКРИТІ ПИТАННЯ (6–10)

6. Поясніть, чому в задачі 1 використовується цикл з кроком 2.

---

7. Як реалізовано пошук рядка в задачі 2?

---

8. Чому сортування вибором має складність $O(n^2)$?

---

9. Які переваги використання файлів для введення/виведення даних?

---

10. Які труднощі можуть виникнути при зміні масиву “на місці” без допоміжного масиву?

Я зроблю це як викладачеві перевірку — **об’єктивно, але з оцінюванням якості відповідей**.

🔷 1) АНАЛІЗ ТЕСТОВИХ ВІДПОВІДЕЙ (1–5)

№	Відповідь	Правильність	Оцінка (5-бальна)	Коментар
1	A	✅	5	правильно обґрунтовано
2	C	✅	5	correct $O(n^2)$
3	C	✅	5	логічно і правильно
4	B	✅	5	відповідно до умови
5	B	✅	5	коректно

📌 Висновок по тестах:

✔ Повна правильність
✔ Розуміння алгоритмів підтверджено

Сума: 25 / 25

🔷 2) АНАЛІЗ ВІДКРИТИХ ВІДПОВІДЕЙ (6–10)

📌 Загальна оцінка якості:

Відповіді:

- правильні
- логічні
- але **дуже узагальнені (типові формулювання з теорії / ШІ-стилю)**

⚠️ Оцінка ймовірності шаблонності (ШІ-стиль)

(не ствердження, а академічна оцінка формулювань)

- 6 → висока узагальненість
- 7 → стандартне формулювання
- 8 → теоретичне визначення
- 9 → типовий опис з лекцій
- 10 → узагальнене інженерне пояснення

👉 Всі відкриті відповіді оцінюються як **висока шаблонність формулювань**

📉 Застосування правила (–60% балів при високій ймовірності шаблонності)

№	Було	Після корекції
6	5	2
7	5	2

8	5	2
9	5	2
10	5	2

📌 Сума відкритих:

10 / 25

🔷 3) ЗАГАЛЬНА ОЦІНКА

📊 Підсумок:

- Тести: **25**
- Відкриті: **10**

🔹 **Разом:**

35 / 50

📌 Середня оцінка (5-бальна шкала):

[
35 ÷ 10 = 3.5
]

🔷 ⭐ ФІНАЛЬНИЙ РЕЗУЛЬТАТ:

👉 **Оцінка: 3.5 / 5 (добре)**

🔷 4) КОМЕНТАР ВИКЛАДАЧА

✔ Код і логіка засвоєні добре
✔ Алгоритми реалізовані правильно
✔ Є розуміння структури програми

⚠ Недолік:

- відкриті відповіді надто загальні
- відсутні приклади або деталізація “своїми словами з коду”
- недостатньо інженерної конкретики (посилання на реалізацію в C++)

🔷 5) ДОДАТКОВО (що ти просив):

📌 ЩО ДОДАТИ ДО ТЕОРІЇ + ПРИКЛАДИ КОДУ

🔹 1. Файлове зчитування (приклад)

```
ifstream fin("array_in.txt");
int n;
fin >> n;

for(int i = 0; i < n; i++) {
   fin >> arr[i];
}
fin.close();
```

📌 Пояснення:

- використовується потоковий ввід

- дані читаються послідовно

🔹 2. Запис у файл

```
ofstream fout("array_out.txt");

fout << n << endl;
for(int i = 0; i < n; i++) {
    fout << arr[i] << " ";
}

fout.close();
```

🔹 3. Альтернатива сортуванню (обмін елементів)

```
for(int i = 0; i < n - 1; i++) {
    for(int j = i + 1; j < n; j++) {
        if(arr[j] < arr[i]) {
            swap(arr[i], arr[j]);
        }
    }
}
```

🔹 4. Обробка матриці (підсилений варіант)

```
int pos = 0, neg = 0;

for(int j = 0; j < n; j++) {
    if(matr[i][j] > 0) pos++;
    else if(matr[i][j] < 0) neg++;
}
```